Project 4: Numerical Integration of Functions with Oscillations

Michael Hwang, Serena Su

Due Date: December 1, 2025

Contents

1 Introduction and Background **3**
1.1 An Intuition and Visualization of Newton-Cotes Formulas . . . 3
1.2 Newton-Cotes Formulas and Simpson's Rule . . . 4

2 Composite Simpson's Rule **5**

3 Simpson's Rule Error **5**
3.1 Local Error . . . 5
3.2 Global Error . . . 6

4 Adaptive Simpson's Rule **8**
4.1 Introduction . . . 8
4.2 Pseudocode . . . 9
4.3 Matlab Implementation . . . 10

5 Q3 - Numerical experiments **10**
5.1 Results . . . 10
5.2 Analysis on Number of Function Evaluations . . . 13

6 Appendix **15**

1 Introduction and Background

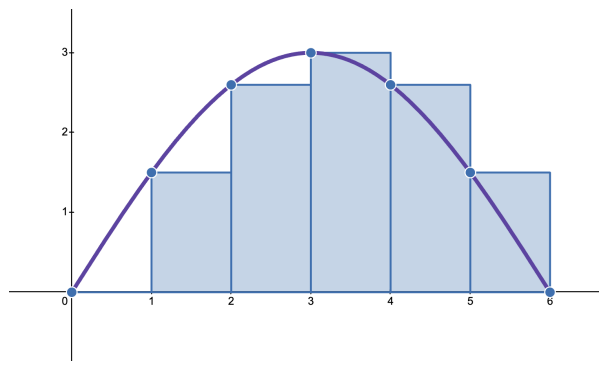
In this section we will provide an introduction to the general Newton-Cotes formulas and then turn our focus to Simpson's Rule.

1.1 An Intuition and Visualization of Newton-Cotes Formulas

In calculus, one learns of Riemann Sums as a method of approximating integrals. Suppose we have some $\mathbb{R}$-valued function f that we want to integrate on the interval $[a, b]$. We can discretize the interval as a partition: $\{a = x_0, x_1, ..., x_n = b\}$, $\Delta x = \frac{b-a}{n}$.

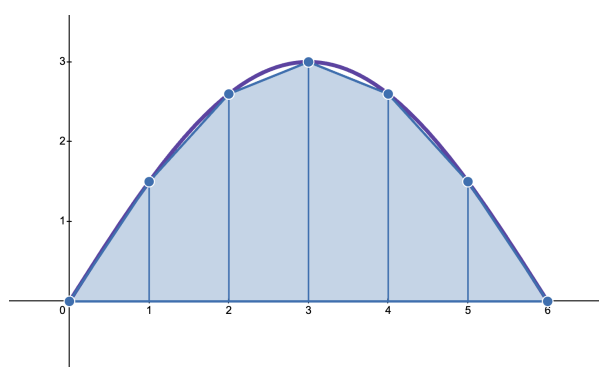
For Riemann Sums, each rectangle is constructed on $[x_{i-1}, x_i]$, $i = 1, 2, ..., n$. Then, the height of each rectangle can be given by f evaluated at the midpoint, or left/right endpoint depending on the scheme. We can approximate the area of each rectangle by multiplying the width, Δx, by the height.

Consider $f(x) = 3\sin\left(\frac{\pi}{6}x\right)$ on $[0, 6]$. We pick the partition $\{0, 1, 2, 3, 4, 5, 6\}$ where $\Delta x = 1$. Below is a graph of a Left-handed Riemann approximation of the integral of f:



Notice how with each rectangle, we are approximating f by a constant function determined by the left-endpoint.

Now, instead of approximating f on each rectangle by a constant function, we instead can use linear functions. Therefore, each rectangle is now a trapezoid whose area formula is known. This gives us the Trapezoid Rule. Here is a graph the Trapezoid approximation of f:

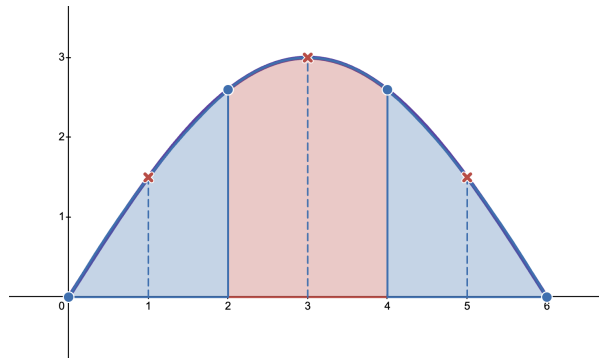


Notice how now in each rectangle, we are approximating f by linear function that is determined by both endpoints.

We can extend this idea to interpolating f by a quadratic function on each rectangle. However, in order to construct an interpolating quadratic, we need 3 nodes in each "rectangle". So instead of each

rectangle being constructed on $[x_{i-1}, x_i]$, it will be constructed on $[x_{2k-2}, x_{2k}]$, $k = 1, ..., n/2$. Note that with this construction, we must have that n be even in order to create our interpolating quadratics on each rectangle. The integral of a quadratic on a closed interval is known. This method of gives us Simpson's Rule.

Here is a Simpson approximation of f:



Notice how now instead of 6 rectangles each with 2 nodes, we have 3 "rectangles" each made up of 3 nodes.

1.2 Newton-Cotes Formulas and Simpson's Rule

The Newton-Cotes Formulas provides the formulas for a single "rectangle" based on the degree of the interpolating polynomial given evenly spaced nodes. The Riemann Sum is an open Newton-Cotes formula of degree 0, open meaning that not all endpoints are included. The Trapezoid and Simpson's Rules are closed (endpoints included) Newton-Cotes formulas of degree 1 and 2 respectively.

In general, on $[a, b]$ with partition $\{a = x_0, x_1, ..., x_n = b\}$ of evenly spaced nodes by $h = \frac{b-a}{n}$. The integral of f can be approximated by the closed Newton-Cotes formula:

$$\int_a^b f(x)\ dx \approx \sum_{i=0}^{n} w_i f(x_i)$$

where the weights w_i are determined by the Lagrange basis polynomials. Also, n is the degree of the interpolating polynomial used in the "rectangle".

More specifically, for $n = 2$, and nodes $\{x_0, x_1, x_2\}$ with step-size $h = \frac{x_2 - x_0}{2}$, Simpson's Rule is given by:

$$\int_{x_0}^{x_2} f(x)\ dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)]$$

Again, note this is our integral approximation of a single "rectangle".

2 Composite Simpson's Rule

We now know Simpson's Rule on a single interval, say some $[x_0, x_2]$. With Composite Simpson's Rule, given a partition $\{a = x_0, x_1, ..., x_n = b\}$ (evenly spaced and n even) of $[a, b]$, we instead use Simpson's Rule on each subdivision $[x_{2k-2}, x_{2k}]$, such that $k = 1, ..., n/2$, and take the sum of the results. This analogous to Riemann Sums where we sum the areas of the rectangles together. Consider some arbitrary $\mathbb{R}$-valued function f and let $h = \frac{b-a}{n}$. If we conduct Simpson's Rule on each of the subdivisions above, we have that:

$$\begin{aligned}
\int_a^b f(x)\ dx &= \int_{x_0}^{x_2} f\ dx + \int_{x_2}^{x_4} f\ dx + ... + \int_{x_{n-2}}^{x_n} f\ dx \\
&\approx \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] + \frac{h}{3}[f(x_2) + 4f(x_3) + f(x_4)] + ... \\
&\quad ... + \frac{h}{3}[f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)] \\
&= \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + ... + 4f(x_{n-1}) + f(x_n)]
\end{aligned}$$

Which gives us the formula for Composite Simpson's Rule:

$$\int_a^b f(x)\ dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + ... + 4f(x_{n-1}) + f(x_n)]$$

Note that all graphs in section 1.1 were generated with the composite version of their respective Newton-Cotes Formulas.

3 Simpson's Rule Error

In this section, we will compute the error of Simpson's rule locally on a single interval and globally with the composite version.

3.1 Local Error

Consider f and nodes $\{x_0, x_1, x_2\}$ with step-size $h = \frac{x_2 - x_0}{2}$. Since the nodes are evenly spaced, note that $x_0 = x_1 - h$ and $x_2 = x_1 + h$. To derive Simpson's Rule along with its corresponding error, we can consider the Taylor expansion of f centered around x_1:

$$f(x) = f(x_1) + f'(x_1)(x - x_1) + \frac{f''(x_1)}{2!}(x - x_1)^2 + \frac{f'''(x_1)}{3!}(x - x_1)^3 + \frac{f^{(4)}(x_1)}{4!}(x - x_1)^4 + \frac{f^{(5)}(x_1)}{5!}(x - x_1)^5 + ...$$

Consider the integral of f on $[x_1 - h = x_0, x_2 = x_1 + h]$

$$\begin{aligned}
\int_{x_0}^{x_2} f(x)\ dx &\approx \int_{x_1-h}^{x_1+h} f(x_1) + f'(x_1)(x - x_1) + \frac{f''(x_1)}{2!}(x - x_1)^2 + \frac{f'''(x_1)}{3!}(x - x_1)^3 \\
&\quad + \frac{f^{(4)}(x_1)}{4!}(x - x_1)^4 + \frac{f^{(5)}(x_1)}{5!}(x - x_1)^5\ dx \\
&= f(x_1)x + f'(x_1)\frac{(x - x_1)^2}{2} + \frac{f''(x_1)}{2!} \cdot \frac{(x - x_1)^3}{3} + \frac{f'''(x_1)}{3!} \cdot \frac{(x - x_1)^4}{4} \\
&\quad + \frac{f^{(4)}(x_1)}{4!} \cdot \frac{(x - x_1)^5}{5} + \frac{f^{(5)}(x_1)}{5!} \cdot \frac{(x - x_1)^6}{6}\Bigg|_{x_1-h}^{x_1+h}
\end{aligned}$$

Denote the RHS as $If(x)|_{x_1-h}^{x_1+h}$. Evaluating $If(x)$ at $x_1 \pm h$:

$$\begin{aligned} If(x_1 \pm h) = f(x_1)(x_1 \pm h) + h^2 \cdot \frac{f'(x_1)}{2} \pm \frac{h^3}{3} \cdot \frac{f''(x_1)}{2!} + \frac{h^4}{4} \cdot \frac{f'''(x_1)}{3!} \\ \pm \frac{h^5}{5} \cdot \frac{f^{(4)}(x_1)}{4!} + \frac{h^6}{6} \cdot \frac{f^{(5)}(x_1)}{5!} \end{aligned}$$

Therefore we have that:

$$\int_{x_0}^{x_2} f(x)\ dx \approx If(x_1+h) - If(x_1-h) = 2hf(x_1) + \frac{h^3}{3} f''(x_1) + \frac{h^5}{60} f^{(4)}(x_1)$$

Where all terms with even powers of h cancel out and we get $2h$ of $f(x_1)$. Approximate $f''(x_1)$ by the center difference scheme:

$$f''(x_1) \approx \frac{f(x_1+h) - 2f(x_1) + f(x_1-h)}{h^2} - \frac{h^2}{12} f^{(4)}(x_1)$$

Substituting this into our integral, we have:

$$\begin{aligned} \int_{x_0}^{x_2} f(x)\ dx &\approx 2hf(x_1) + \frac{h^3}{3}\left[\frac{f(x_1+h) - 2f(x_1) + f(x_1-h)}{h^2} - \frac{h^2}{12} f^{(4)}(x_1)\right] + \frac{h^5}{60} f^{(4)}(x_1) \\ &= 2hf(x_1) + \frac{h^3}{3}\left[\frac{f(x_1+h) - 2f(x_1) + f(x_1-h)}{h^2}\right] - \frac{h^5}{36} f^{(4)}(x_1) + \frac{h^5}{60} f^{(4)}(x_1) \\ &= \frac{6}{3} hf(x_1) + \frac{h}{3}\left[f(x_1+h) - 2f(x_1) + f(x_1-h)\right] + \left(\frac{h^5}{60} - \frac{h^5}{36}\right) f^{(4)}(x_1) \\ &= \frac{h}{3}\left[f(x_1-h) + 4f(x_1) + f(x_1+h)\right] - \frac{h^5}{90} f^{(4)}(x_1) \\ &= \frac{h}{3}\left[f(x_0) + 4f(x_1) + f(x_2)\right] + O(h^5) \end{aligned}$$

Therefore locally, Simpson's rule is $O(h^5)$ with truncation error $-\frac{h^5}{90} f^{(4)}(\xi)$, for some $\xi \in (x_0, x_2)$.

3.2 Global Error

Now consider f on the interval $[a, b]$ with evenly spaced partition $\{a = x_0, x_1, ..., x_n = b\}$ such that $h = \frac{b-a}{n}$ and n is even. We have Composite Simpson's Rule as:

$$\int_a^b f(x)\ dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + ... + 4f(x_{n-1}) + f(x_n)]$$

On each sub-interval $[2k-2, 2k]$, $k = 1, ..., n/2$, we found that the local error was:

$$E_k = -\frac{h^5}{90} f^{(4)}(\xi_k), \text{ for some } \xi_k \in (x_{2k-2}, x_{2k})$$

Therefore, for global error, we can take the sum of the local errors.

$$E_{global} = \sum_{k=1}^{n/2} E_k = \sum_{k=1}^{n/2} -\frac{h^5}{90} f^{(4)}(\xi_k) = -\frac{h^5}{90} \sum_{k=1}^{n/2} f^{(4)}(\xi_k)$$

Assuming $f \in C^4[a, b]$, suppose that:

$$m = \min_{x \in [a,b]} f^{(4)}(x), \quad M = \max_{x \in [a,b]} f^{(4)}(x)$$

Note that since $f^{(4)}$ is continuous on $[a, b]$, both m and M are attained by f. Suppose $f^{(4)}(a') = m$, $f^{(4)}(b') = M$, and without loss of generality, assume $a' \leq b'$. Note that we must have $f^{(4)} \in C[a', b']$.

For each sub-interval $[2k-2, 2k]$, we have:

$$m \leq f^{(4)}(\xi_k) \leq M$$

Taking the sum over all $n/2$ sub-intervals:

$$\frac{n}{2} \cdot m \leq \sum_{k=1}^{n/2} f^{(4)}(\xi_k) \leq \frac{n}{2} \cdot M$$

$$m \leq \frac{2}{n} \sum_{k=1}^{n/2} f^{(4)}(\xi_k) \leq M$$

Note that $n/2 > 0$.

By the Intermediate Value Theorem, since $f^{(4)} \in C[a', b']$ and $f(a') = m \leq \frac{2}{n} \sum_{k=1}^{n/2} f^{(4)}(\xi_k) \leq M = f(b')$, there exists some $\xi \in (a', b')$ such that:

$$f^{(4)}(\xi) = \frac{2}{n} \sum_{k=1}^{n/2} f^{(4)}(\xi_k)$$

$$\frac{n}{2} \cdot f^{(4)}(\xi) = \sum_{k=1}^{n/2} f^{(4)}(\xi_k)$$

Note that if $m = M$, then $f^{(4)}$ is necessarily constant and IVT would not be needed.

Also note $\xi \in (a', b') \subset (a, b)$ implies that $\xi \in (a, b)$. Therefore, we have the global error:

$$E_{global} = -\frac{h^5}{90} \sum_{k=1}^{n/2} f^{(4)}(\xi_k) = -\frac{h^5}{90} \cdot \frac{n}{2} \cdot f^{(4)}(\xi), \quad \xi \in (a, b)$$

Since $h = \frac{b-a}{n}$, we have that $n = \frac{b-a}{n}$ and:

$$E_{global} = -\frac{h^5}{180} \cdot \frac{b-a}{h} \cdot f^{(4)}(\xi) = -\frac{h^4(b-a)}{180} f^{(4)}(\xi), \quad \xi \in (a, b)$$

Therefore the global error is $O(h^4)$ with truncation error $-\frac{h^4(b-a)}{180} f^{(4)}(\xi)$.

4 Adaptive Simpson's Rule

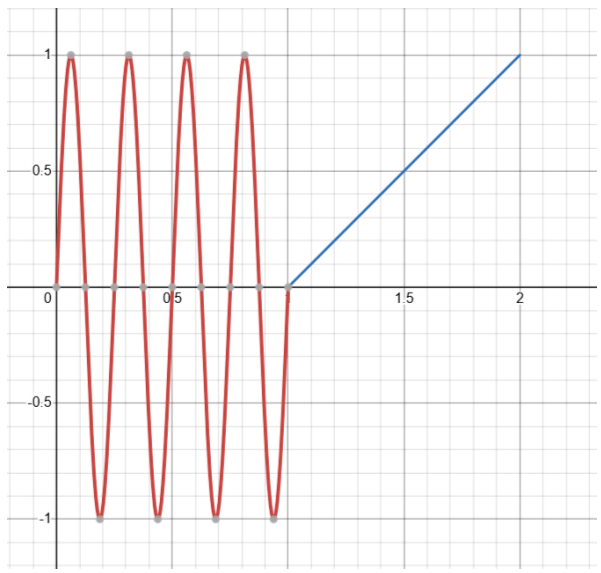
4.1 Introduction

Adaptive Simpson's Rule is a modification of Composite Simpson's Rule. Rather than dividing the entire domain by evenly spaced nodes, Adaptive Simpson's recursively applies Simpson's Rule on each sub-interval until a certain tolerance is met.

Let us denote Simpson's Rule on the interval $[a, b]$ as $S(a, b)$ with the nodes $\{a, \frac{b-a}{2}, b\}$. This method first starts by finding $S(a, b)$ and checking if our approximation is within a certain tolerence.

If not, then we find Simpson's on each half: $S(a, \frac{b-a}{2})$, $S(\frac{b-a}{2}, b)$, and take their sum as our new approximation. If this approximation is within the tolerance we are done. If not, then recursively repeat this process.

For example, consider the function below.



If we integrate the function from 0 to 2, the composite Simpson's rule is exact on the subinterval $[1, 2]$. Thus, any refinement on $[1, 2]$ does not improve the approximation. However, on the subinterval $[0, 1]$, we definitely need more nodes to get a reasonable approximation. So the adaptive Simpson's rule in this case will have $h = 1$ on $[1, 2]$ and $h < 1$ on $[0, 1]$. Therefore our node choice will not necessarily be evenly spaced throughout the domain.

This example illustrates that the adaptive Simpson's rule can be more computationally efficient than the composite Simpson's rule, which equally divides subintervals.

4.2 Pseudocode

Listing 1: Pseudo code

```
Function recurSimpson(f, a, b, tol, F)
    # f  = integrand
    # F  = exact antiderivative of f
    # tol = local tolerance on [a,b]

    ------------------------------------------------------------
    # Step 1: Compute basic Simpson rule on [a,b]
    ------------------------------------------------------------
    h = (b - a) / 2
    m = a + h

    fa = f(a)
    fm = f(m)
    fb = f(b)

    I  = (h / 3) * (fa + 4*fm + fb)
    evalCount = 3                # Three evaluations of f


    ------------------------------------------------------------
    # Step 2: Compute exact integral for error checking
    ------------------------------------------------------------
    I_exact = F(b) - F(a)
    error   = | I - I_exact |

    ------------------------------------------------------------
    # Step 3: If interval is accurate enough or the step size
    is less than 1e-12, return results
    ------------------------------------------------------------
    If error < tol OR h < 1e-12 then
        return (I, evalCount)
    End If

    ------------------------------------------------------------
    # Step 4: Otherwise subdivide into two halves
    ------------------------------------------------------------
    (I1, E1) = recurSimpson(f, a, m, tol/2, F)
    (I2, E2) = recurSimpson(f, m, b, tol/2, F)

    ------------------------------------------------------------
    # Step 5: Combine results from children
    ------------------------------------------------------------
    I_total  = I1 + I2
    evals    = evalCount + E1 + E2

    return (I_total, evals)
End Function
```

4.3 Matlab Implementation

Listing 2: Matlab code

```
function [I, N_eval] = recurSimpson(f, a, b, tol, F)
    % Adaptive Simpson using exact antiderivative and counting function evals.
    %
    % OUTPUTS:
    %   I        - integral estimate
    %   N_eval   - total number of f(x) evaluations

    % --- Step 1: compute Simpson panel on [a,b] ---
    h = (b - a) / 2;
    m = a + h;

    % Evaluate f(a), f(m), f(b) and count them
    fa = f(a);
    fm = f(m);
    fb = f(b);
    N_eval = 3;      % 3 function evaluations on this interval

    % Simpson rule
    I = (h/3) * (fa + 4*fm + fb);

    % Exact integral for error check
    I_exact = F(b) - F(a);

    % --- Step 2: accept interval if accurate ---
    if abs(I - I_exact) < tol || h < 1e-12
        return
    end

    % --- Step 3: otherwise subdivide and recurse ---
    [I1, E1] = recurSimpson(f, a, m, tol/2, F);
    [I2, E2] = recurSimpson(f, m, b, tol/2, F);

    % Combine child results
    I        = I1 + I2;
    N_eval   = N_eval + E1 + E2;
end
```

5 Q3 - Numerical experiments

We perform numerical experiments for each of the following functions:

3a). $f_1(x) = e^{-x}$ on $[0, 1]$

3b). $f_2(x) = \sin(101x)$ on $[0, \pi]$

3c). $f_3(x) = x^{-1/2}$ on $[0, 1]$

5.1 Results

As the graphs of the functions show (Figure (1)), f_1 is smooth and uniformly continuous, f_2 shows high oscillations, and f_3 goes to infinity near 0. So we can expect that the adaptive Simpson's rule converges quickly for f_1 but more slowly for f_2 and f_3.

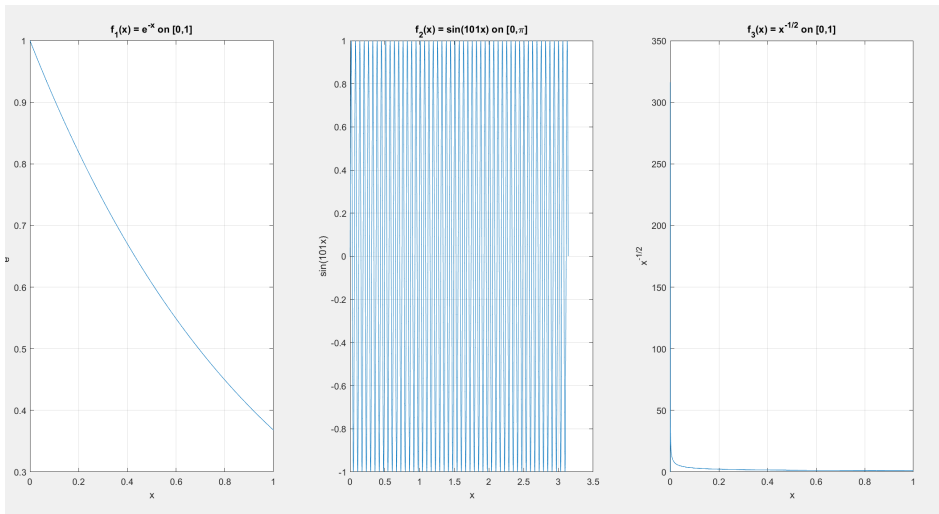


Figure 1: Graphs of the Functions

	Tolerance $= 10^{-4}$	10^{-6}	10^{-8}
f_1	0.632134175320532	0.632121133883837	0.632120562177264
f_2	0.0198019431620008	0.0198019858781805	0.0198019802382834
f_3	2.00003365077696	1.99999913239999	1.99999880295925

Table 1: Approximate Integrals

	Tolerance $= 10^{-4}$	10^{-6}	10^{-8}
f_1	1.36E-05	5.75E-07	3.34E-09
f_2	3.70E-08	5.68E-09	4.02E-11
f_3	3.36E-05	8.67E-07	1.19E-06

Table 2: Estimated Errors

	Tolerance $= 10^{-4}$	10^{-6}	10^{-8}
f_1	9	27	93
f_2	4785	11853	44997
f_3	2871	7935	21909

Table 3: Number of Function Evaluations

	f_1	f_2	f_3
Exact Integral	$\frac{e-1}{e} \approx 0.63212$	$\frac{2}{101} \approx 0.019802$	2

Table 4: Exact Integral

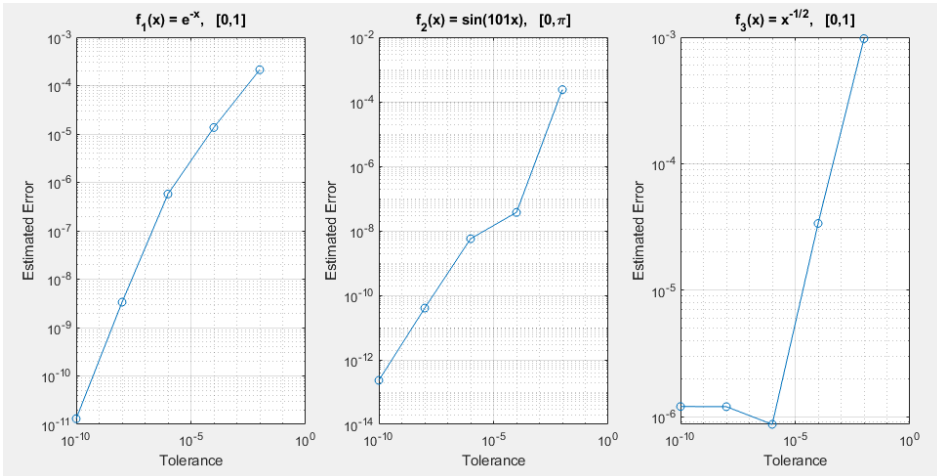


Figure 2: Estimated Error vs Tolerance (log-log)

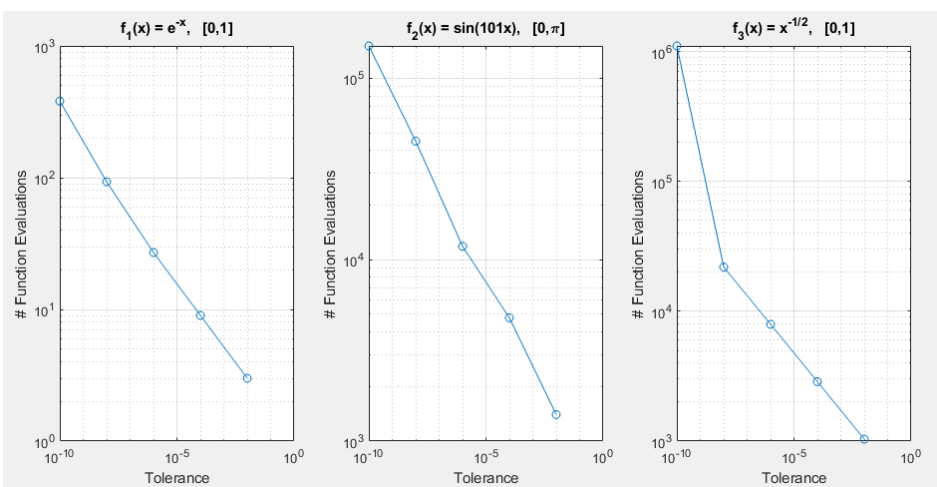


Figure 3: Number of Function Evaluations vs Tolerance (log-log)

From the above results, we see that the adaptive Simpson's rule is suitable only for the smooth and uniformly continuous function, f_1: We can achieve the accuracy of 10^{-9} with 93 function evaluations. However, to attain the same accuracy, the adaptive Simpson's rule for f_2 and f_3 requires more than 10,000 function evaluations.

5.2 Analysis on Number of Function Evaluations

Note that in the worst case scenario, Adaptive Simpson's would behave exactly like Composite Simpson's. In this section, we will find the number of function evaluations required for each f to reach tolerance with Composite Simpson's. Then, we will compare this to the results from our implementation of Adaptive Simpson's.

We can analyze this behavior using the global error bound from the previous part:

$$E_{global} = -\frac{b-a}{180} h^4 f^{(4)}(\xi)$$

The fourth derivatives of the functions are:

$$f_1^{(4)} = e^{-x} \tag{1}$$

$$f_2^{(4)} = 101^4 \sin(101x) \approx 10^8 \sin(101x) \tag{2}$$

$$f_3^{(4)} = \frac{105}{16} x^{-9/2} \tag{3}$$

Thus, we get the error bound for each function as follows:

$$|E_1| = \left| -\frac{1-0}{180} h^4 e^{-x} \right| \leq \frac{e}{180} h^4 \leq h^4 \tag{4}$$

$$|E_2| = \left| -\frac{\pi - 0}{180} h^4 101^4 \sin(101x) \right| \approx 10^8 h^4 \tag{5}$$

$$|E_3| = \left| -\frac{1-0}{180} h^4 \frac{105}{16} x^{-9/2} \right| \leq 0.04 \cdot x^{-9/2} h^4 \tag{6}$$

Using the above error bounds, we can approximate the number of function evaluations required to attain the accuracy of 10^{-8} for f_1 and f_2.

For f_1, we have

$$|E_1| \leq h^4 = 10^{-8} \iff h = 10^{-2}.$$

Since Step 4 in the pseudo code (recursive function call) divides the interval by half, to get $h = 10^{-2}$, we need

$$(b-a)\left(\frac{1}{2}\right)^n = 1\left(\frac{1}{2}\right)^n < 10^{-2} \iff n \geq 7,$$

where n is the number which Step 4 is executed. So the number of function evaluations in this case is roughly,

$$3 \cdot \sum_{k=0}^{7} 2^k \approx 3 \int_0^7 2^x \, dx = 3 \cdot \ln(2) \cdot 2^7 \approx 266,$$

where the experimental result is 93.

For f_2, we have

$$|E_2| \leq 10^8 h^4 = 10^{-8} \iff h = 10^{-4}.$$

Thus, the number n, for which Step 4 is executed in this case, is

$$(\pi - 0)\left(\frac{1}{2}\right)^n = \pi\left(\frac{1}{2}\right)^n < 10^{-4} \iff n \geq 15.$$

So the number of function evaluations in this case is roughly,

$$3 \cdot \sum_{k=0}^{15} 2^k \approx 3 \int_0^{15} 2^x \, dx = 3 \cdot \ln(2) \cdot 2^{15} \approx 68139,$$

where the experimental result is 4785. The point is that f_2 is not suitable for the adaptive Simpson's rule because of the factor 10^8 in the error bound.

In the case of f_3, we see the the error bound goes to infinity near 0 because of the factor $x^{-9/2}$.

Conclusion: The adaptive Simpson's rule is suitable only for $f_1 = e^{-x}$, which is smooth and uniformly continuous on $[0, 1]$.

6 Appendix

Below is the complete code for reference.

Listing 3: Numerical experiments

```
% Numerical experiment for adaptive Simpson's rule

%% -------------------------------------------------------
% 1. Define test integrands f_1, f_2, f_3
%% -------------------------------------------------------

f = cell(1,3);

% f1(x) = e^{-x}
f{1} = @(x) exp(-x);

% f2(x) = sin(101x) (high oscillation)
f{2} = @(x) sin(101*x);

% f3(x) = x^{-1/2},
% (piecewise: 0 for x<=0, and 1/sqrt(x) for x>0)
f{3} = @(x) (x > 0) .* (1 ./ sqrt(max(x, eps)));


%% -------------------------------------------------------
% 2. Exact antiderivatives (for the estimated error)
%% -------------------------------------------------------

F = cell(1,3);

% Integral of e^{-x} is -e^{-x}
F{1} = @(x) -exp(-x);

% Integral of sin(101x) is -(1/101) cos(101x)
F{2} = @(x) -1/101 * cos(101*x);

% Integral of x^{-1/2} is 2 sqrt(x)
F{3} = @(x) 2 * sqrt(x);


%% -------------------------------------------------------
% 3. Integration intervals for each test function
%% -------------------------------------------------------

endPoints = [0 1;   % for f1
             0 pi;  % for f2
             0 1];  % for f3


%% -------------------------------------------------------
% 4. Tolerances for adaptive Simpson
%% -------------------------------------------------------

power = (-2:-2:-10);    % -2, -4, ..., -10
Tol = 10.^power;        % Tols = 1e-2, 1e-4, ..., 1e-10


%% -------------------------------------------------------
% 5. Storage for results
```

```
%% -------------------------------------------------------

I       = zeros(3, length(Tol));    % approximate integrals
N_eval = zeros(3, length(Tol));    % number of function evaluations

% Exact integral values on the given intervals
I_exact = [(exp(1)-1)/exp(1) 2/101 2];


%% -------------------------------------------------------
% 6. Run adaptive Simpson for each function & tolerance
%% -------------------------------------------------------

for i = 1:3                      % loop over test functions
    for j = 1:length(Tol)       % loop over tolerances
        [I(i,j), N_eval(i,j)] = ...
            recurSimpson(f{i}, endPoints(i,1), endPoints(i,2), Tol(j), F{i});
    end
end


%% -------------------------------------------------------
% 7. Compute absolute error for each test
%% -------------------------------------------------------

estimate_error = zeros(3, length(Tol));
for i = 1:3
    estimate_error(i,:) = abs(I(i,:) - I_exact(i));
end


%% -------------------------------------------------------
% 8. Plot: Estimated error vs tolerance (log-log)
%% -------------------------------------------------------

figure;

subplot(1, 3, 1);
loglog(Tol, estimate_error(1,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('Estimated Error');
title('f_1(x) = e^{-x},   [0,1]');

subplot(1, 3, 2);
loglog(Tol, estimate_error(2,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('Estimated Error');
title('f_2(x) = sin(101x),   [0,\pi]');

subplot(1, 3, 3);
loglog(Tol, estimate_error(3,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('Estimated Error');
title('f_3(x) = x^{-1/2},   [0,1]');


```

```
%% -------------------------------------------------------
% 9. Plot: Number of function evaluations vs tolerance
%% -------------------------------------------------------

figure;

subplot(1, 3, 1);
loglog(Tol, N_eval(1,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('# Function Evaluations');
title('f_1(x) = e^{-x},   [0,1]');

subplot(1, 3, 2);
loglog(Tol, N_eval(2,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('# Function Evaluations');
title('f_2(x) = sin(101x),   [0,\pi]');

subplot(1, 3, 3);
loglog(Tol, N_eval(3,:), '-o');
grid on;
xlabel('Tolerance');
ylabel('# Function Evaluations');
title('f_3(x) = x^{-1/2},   [0,1]');


%% -------------------------------------------------------
% 10. Plot the integrands (for visualization)
%% -------------------------------------------------------

% Fine grid for each interval
xx = [linspace(0,1, 1e5);
      linspace(0,pi,1e5);
      linspace(0,1, 1e5)];

% Evaluate each function on its interval
yy = [f{1}(xx(1,:));
      f{2}(xx(2,:));
      f{3}(xx(3,:))];

figure;

subplot(1, 3, 1);
plot(xx(1,:), yy(1,:), '-');
grid on;
xlabel('x');
ylabel('e^{-x}');
title('f_1(x) = e^{-x} on [0,1]');

subplot(1, 3, 2);
plot(xx(2,:), yy(2,:), '-');
grid on;
xlabel('x');
ylabel('sin(101x)');
title('f_2(x) = sin(101x) on [0,\pi]');

subplot(1, 3, 3);
```

```
plot(xx(3,:), yy(3,:), '-');
grid on;
xlabel('x');
ylabel('x^{-1/2}');
title('f_3(x) = x^{-1/2} on [0,1]');
```